==========

POSTMORTEM INC-5102

orders-api pods OOMKilled under retry storm; order submission degraded 3h47m

==========

jira : INC-5102 (postmortem PLAT-884; actions PLAT-885..889)
severity : SEV-2
environment : production / cluster eks-prod-eu-1 / namespace orders
component : orders-api v2026.7.3, redis-orders 7.2, kong-gw 3.4
opened : 2026-08-03 14:22 UTC
mitigated : 2026-08-03 16:41 UTC
closed : 2026-08-03 18:09 UTC
author : k.oyelaran
reviewers : d.fischer, p.almeida, s.hartmann
status : closed, all actions shipped

---

1. SUMMARY

Between 14:22 and 18:09 UTC on 3 August 2026, order submission through orders-api failed intermittently, peaking at 34% of POST /v2/orders returning 503. Pods in the orders-api deployment were killed by the kernel OOM killer in a repeating cycle: a pod would accumulate heap until it hit its 1.5 GiB memory limit, be killed, and its in-flight requests would be retried onto the remaining pods, accelerating their own growth. 96 pod restarts were recorded in the window against a baseline of zero.

Read paths (GET /v2/orders, order history, tracking) were unaffected because they are served by a separate deployment. No orders were lost: every rejected submission returned 503 with a client-visible retry hint, and the idempotency key store prevented duplicates on retry.

---

2. IMPACT

---

failed submissions : 21,904 of 64,410 attempts (34.0% peak, 12.1% mean)
customers affected : ~8,100 unique accounts
orders permanently lost : 0 (all retried successfully within 15 min)
revenue at risk : CHF 1.29m in basket value, none confirmed lost
slo burn : 41% of the monthly error budget for orders-api
support tickets : 212

orders-api pod restarts per hour, 2026-08-03 UTC

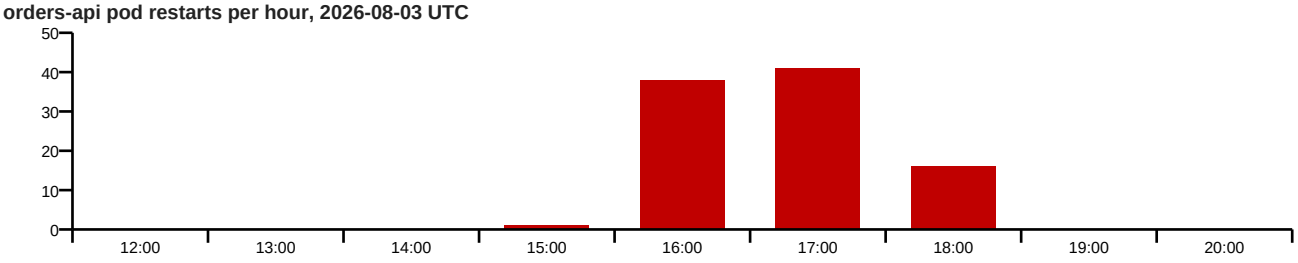

Figure 1 — restarts per hour. Baseline for this deployment is zero restarts per day.

Heap in use, MiB - affected pod (orange) vs control pod (teal)

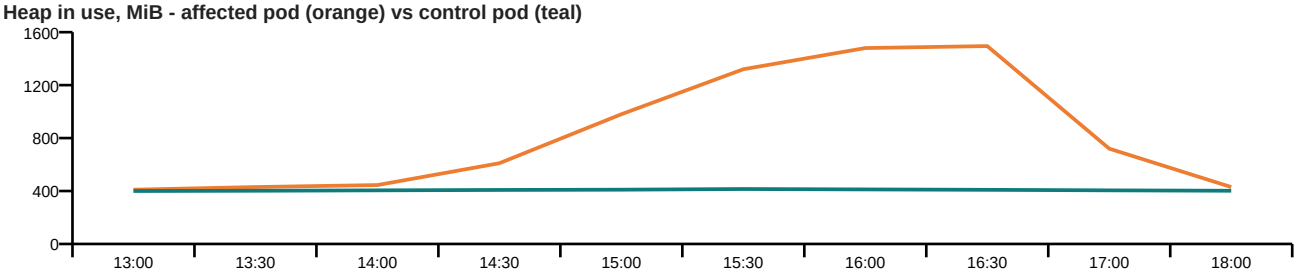

Figure 2 — the affected pod's heap grows monotonically to the 1,536 MiB limit. The control pod, serving the same traffic with the cache disabled, is flat.

---

3. TIMELINE (UTC)

```
2026-07-30 11:05 orders-api v2026.7.3 released. Ships a new in-process
response cache for the pricing lookup (PR #4412).
2026-08-03 09:00 Weekend promo campaign begins; submission volume ~2.4x
the weekday baseline.
2026-08-03 14:22 First OOMKill on orders-api-7c9d4. Alert
ORDERS-POD-RESTART fires (threshold: >2 restarts/10m).
2026-08-03 14:25 k.oyelaran acknowledges. Dashboards show heap climbing
linearly on every pod since the 30 July release.
2026-08-03 14:40 Declared SEV-2. p.almeida joins as ops.
2026-08-03 14:58 Memory limit raised 1.5 -> 3.0 GiB as a holding action.
This slows the cycle but does not stop it.
2026-08-03 15:20 Heap dump captured from orders-api-7c9d4 and analysed.
90% of live objects are PricingCacheEntry.
2026-08-03 15:44 Cause confirmed: the cache is unbounded and keyed by a
value that includes the request id, so it never hits.
2026-08-03 16:05 Change PLAT-884-hotfix approved: disable the cache via
the PRICING_CACHE_ENABLED feature flag.
2026-08-03 16:41 Flag flipped to false; rolling restart completed.
Error rate returns to baseline. Incident mitigated.
2026-08-03 18:09 90 minutes of flat heap observed on all pods.
Incident closed.
2026-08-05 10:00 Postmortem review with platform-eng and payments.
```

---

4. ROOT CAUSE

The pricing response cache added in PR #4412 was constructed with an unbounded `HashMap` and its key was built from the full request context object, whose `toString()` includes the per-request correlation id. Every request therefore produced a unique key: the cache never returned a hit and never evicted an entry. It was, in effect, a memory leak with a cache-shaped API. Under normal weekday volume a pod reached its 1.5 GiB limit in roughly 40 hours, which is longer than the deployment's usual lifetime, so the leak was invisible in the four days between release and incident. The promo campaign on 3 August tripled request volume and cut that time to about five hours, at which point all pods began hitting the limit together.

The retry storm that turned a rolling restart into a 34% failure rate is a second-order effect: kong-gw retried 503s up to three times with no jitter, so each OOMKill produced a synchronised burst of retries onto the surviving pods.

---

5. ACTION PLAN

Jira	Action	Owner	Due	Status
PLAT-885	Replace the pricing cache with a bounded Caffeine cache (max 50k entries, 10 min TTL) keyed by sku+currency+tier only.	k.oyelaran	2026-08-10	Done
PLAT-886	Add a soak stage to the release pipeline: 60 minutes at 3x peak traffic, failing the release if RSS grows more than 5%.	d.fischer	2026-08-24	Done
PLAT-887	Alert on heap growth trend (>10%/hour sustained for 2 hours) rather than only on restarts, so the leak is caught before the kill.	p.almeida	2026-08-17	Done
PLAT-888	Add jittered exponential backoff to kong-gw retries and cap retry budget at 10% of request volume.	p.almeida	2026-08-21	Done
PLAT-889	Require a cache-design note (bound, eviction, key, expected hit rate) in the PR template for any new in-process cache.	d.fischer	2026-08-14	Done

6. LESSONS

- A cache with a 0% hit rate is indistinguishable from a leak, and nothing in our metrics reported cache hit rate.
- Raising the memory limit bought time but hid the signal; we should have taken the heap dump first.
- Load tests run at 1x peak for 10 minutes. Neither the duration nor the multiple would have surfaced this.